

APEX Autopilot

Autonomous Prediction-Market Execution Engine

AI-governed cross-venue arbitrage for Kalshi × Polymarket — with a 14-check risk stack, multi-agent intelligence, and Bloomberg-style operator terminal.

100+

Live Arb Opportunities

L0–L4

Architecture Layers

260 Days

Daily Execution Plan

M01–M09

Risk Gates

Human traders miss speed, context, and discipline.

■ Fragmented Data

Kalshi order books, Polymarket CLOB, settlement language, macro feeds — each in a different silo.
Edge is gone by reconciliation time.

■ Execution Lag

Real arb windows: seconds, not minutes.
Spreadsheets and Discord alerts are structurally too slow to capture cross-venue structural spreads.

■ Inconsistent Risk

No deterministic gates → policy drift.
One trader sizes aggressively, another skips settlement checks.
Post-trade forensics become arguments.

Result: alpha decay · avoidable blow-ups · non-reproducible decisions

Cross-venue prediction markets need execution infrastructure.

Regulated & on-chain venues

Kalshi (CFTC-regulated) and Polymarket both reached exchange-grade volumes. Event-driven derivatives are no longer a curiosity — they're an asset class.

The structural arb wedge

The same economic proposition priced differently across venues, after fees, settlement alignment, and executable depth. High-frequency in time; low in competent operators.

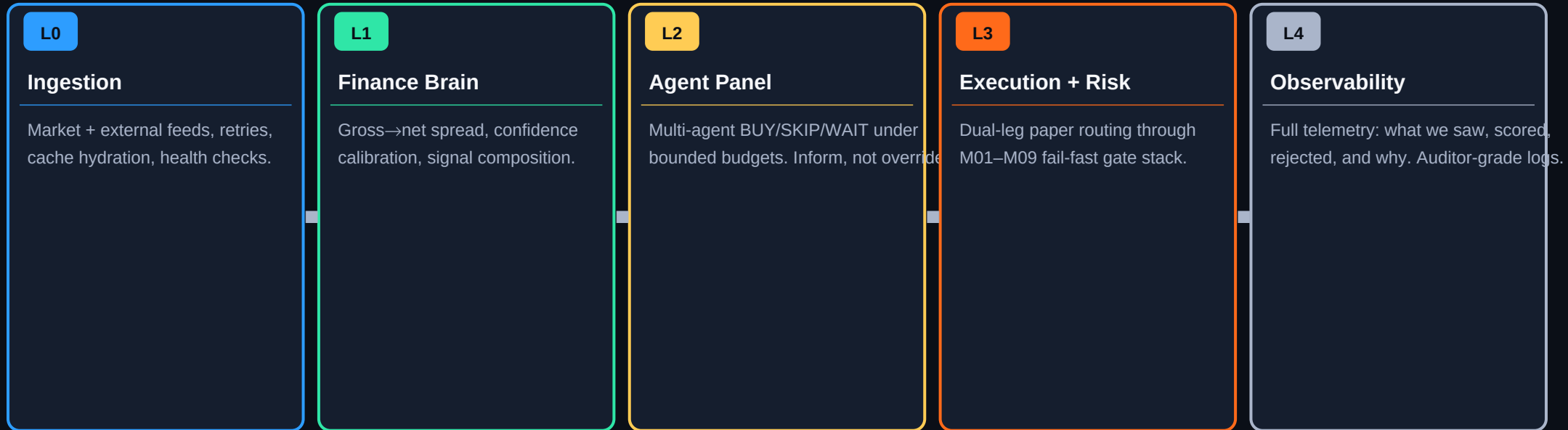
Execution = the moat

Spotting a spread is commoditised. Proving you can execute both legs safely, with an audit trail, is not. That's the wedge we're attacking.

260-day runway ahead

APEX compounds a data flywheel from every rejected trade: calibration, settlement precision, model scoring — structural advantages that grow over time.

A controlled autonomous engine — not a black box.



Architecture is the moat. The terminal is the window.

Bloomberg-style terminal for autonomous decision support.

■ Arb Radar

Ranked opportunities with net edge,
confidence score, settlement alignment.
Refreshed from live SQLite arb store.
Colour-coded priority tiers.

■ Autopilot Console

Live proposals, execution lifecycle,
backend health indicators.
Real-time gate pass/fail visibility.
Paper-enforcement status badge.

■ Intelligence Layer

External source checks, news risk,
consensus overlays (Bright Data-backed).
Soft-fail: optional services never
take down core trading flow.

■ Operator Controls

Paper enforcement toggle.
Token-gated high-impact actions.
Deterministic fallbacks on LLM/API fail.
Full audit stream replay.

Running today: real APIs, real gates, real terminal.

Capability	Status	Evidence
Arb opportunity generation	■ LIVE	/api/arb/opportunities — SQLite-backed, refreshed
Active proposals	■ LIVE	/proposals — wired to operator terminal
M01–M09 risk gates	■ ENFORCED	RiskEngine.run_arb_paper() — M01-first, fail-fast
Next.js operator terminal	■ LIVE	autopilot-local/frontend + Playwright E2E
Intelligence pipeline	■ TUNING	Agent verdict flows, cron jobs, Bright Data adapter
103+ unit tests	■ ACTIVE	pytest tests/ — critical path coverage
Backtest / settlement	■ ROADMAP	backtest_engine.py, settlement_auditor.py

M01 PAPER_REQUIRED runs first on every single arb execution path — without exception.

Compound moat: architecture + culture + data flywheel.

■ Safety-First by Design

Paper defaults and deterministic gate ordering prevent the class of accidents that kill fintech companies early. M01 is not a checkbox — it is the contract.

■ Cross-Market Intelligence

We combine microstructure signals with verified external data in one structured context. Not a chat thread. Not a prompt — a typed decision pipeline with rollback.

■ Operator Trust

Transparent logs and deterministic fallbacks mean the system is debuggable at 2 a.m. Operators see every gate decision, every rejection reason, every verdict.

■ Data Flywheel

Every rejected trade is training data: calibration, settlement precision, model scoring. Week 4+ ML loops compound this advantage against anyone starting today.

Competitors can copy a dashboard. They cannot quickly copy culture + architecture + runbooks.

260 days: paper alpha → production-ready autonomy.



Terminal → Team → Institutional.

Monetisation follows operator trust. We charge for infrastructure and control — not signals.

Pro Terminal	Team Ops	Institutional	Performance (Future) Rev-share
\$500–\$2k/mo	\$5–\$15k/mo	Custom	Rev-share
› Arb radar › Intelligence overlays › Simulation analytics › 1 seat	› Policy controls › Audit log exports › Strategy workspaces › 5–20 seats	› Dedicated VPC deploy › SSO + custom gates › Risk policy frameworks › SLA + white-glove	› Live capital paths only › Compliance-gated › After Year 2+ › Venue approval req'd

APEX wins where execution + risk + PM-specific matching meet.

Alternative	Their Gap	APEX Advantage
Manual + scripts	No unified risk; no audit stream	Full M01–M09 gate stack + SSE logs
Generic LLM agents	No deterministic gates; no dual-leg	Agents inform; gates decide
Single-venue tools	Miss cross-platform arb	Kalshi × Polymarket dual-leg routing
Traditional EMS	Not built for PM settlement semantics	Typed settlement-match scoring (M05)
Ad-hoc quant scripts	No operator UX; no observability	Bloomberg-style terminal + L4 telemetry

Built by practitioners — shipped daily, not Figma-first.

Team Autopilot — markets × infra × ML × product.



Aarav J.

Founder & Lead Engineer

Full L0–L4 stack architect.
260-day daily runbook author.
Kalshi + Polymarket API integrations.



[Quant]

Quantitative Modeler

Market microstructure.
Calibration, risk metrics, Kelly/VaR.
ML scoring pipeline (Week 4+ track).



[Infra]

Platform Engineer

FastAPI, Redis, job scheduling.
Docker/CI deployment hardening.
Observability + test coverage.



[Product]

Product & Ops

Operator UX, runbooks.
Design partner onboarding.
PM settlement graph research.

THE ASK

Partner to accelerate engine → platform.

\$[X]

Raise (SAFE / Priced)

12–18 mo

Runway

3 Roles

Engineering Hires

2 Pilots

Design Partners

USE OF FUNDS

40%

Engineering Reliability

CI, observability, startup hardening, test coverage depth

35%

Quant + Signal Quality

Settlement, backtest, ML scoring, execution state machine

25%

GTM + Design Partners

Terminal polish, pilot onboarding, docs, partner success

HIRING PLAN

Platform/Infra — FastAPI, Redis, deployment

Quant — microstructure, calibration, risk metrics

Product Ops — operator UX, partner onboarding

Autonomy with discipline

wins this market.

Building the operating system for intelligent prediction-market execution.

Product Demo

Technical Diligence

Pilot Planning

APEX Autopilot

Team Autopilot · Prediction-Market Execution Infrastructure

- **GitHub** `github.com/aaravjj2/Autopilot-public`
- **License** Apache 2.0 — open source
- **Demo Mode** `export DEMO_MODE=true && python scripts/seed_demo.py`
- **Start All** `bash start_all.sh → http://localhost:8000/health`
- **Email** `[your-email@domain.com]`

✓ PAPER-ONLY MODE ENFORCED (M01-FIRST)

M01–M09: Sequential, Fail-Fast, Logged.

M00	Valid Opportunity	Tickers / market IDs present and parseable
M01	PAPER ONLY ★	Alpaca paper + Polymarket paper flags required — runs FIRST
M02	Net Edge Floor	Minimum net spread after fees must exceed threshold
M03	24h Volume Floors	Both Kalshi and Polymarket legs must meet volume minimums
M04	Price Sanity	Asks must be in valid probability range [0.01, 0.99]
M05	Settlement Match	Blocks contracts with misaligned resolution language (NLP score)
M06	Daily Arb Loss Cap	Cumulative daily paper P&L cannot exceed configured drawdown
M07	Liquidity / VWAP	Depth must support 3× stake; VWAP slippage on both legs checked
M08	Spread Width	Kalshi orderbook spread must be within executable bounds
M09	Kelly + CFTC Cap	Fractional Kelly sizing capped by CFTC notional position limit

First failure stops execution. Rejection reason is always logged to L4 audit stream.

Core services and endpoints.

GET	/health	System health — all dependency statuses
GET	/api/arb/opportunities	Ranked arb list (canonical; bind UI here)
GET	/proposals	Active trade proposals with lifecycle state
GET	/api/arb/opportunities/:id	Single opportunity detail + gate trace
SSE	/stream	L4 observability stream — real-time operator events

KEY MODULES

arb_engine.py Scan, rank, persist opportunities	settlement_auditor.py Outcome resolution & precision	backtest_engine.py Historical performance replay
risk_checks.py M01–M09 gate implementations	sqlite_store.py All persistence (append-only)	observability.py L4 audit log + SSE stream